Exp 3

Aim :- Create a Cryptocurrency using Python and perform mining in the Blockchain created.

Theory :- A

Blockchain Overview

A blockchain is a distributed, immutable ledger composed of blocks linked via cryptographic hashes. Each block contains a hash of the previous block, a timestamp, and transaction data—typically organized using a Merkle tree. This structure ensures that altering any block requires changing all subsequent blocks and achieving network-wide consensus, making tampering practically infeasible.

Blockchains operate on peer-to-peer (P2P) networks where nodes collectively maintain the ledger. They are categorized into public (permissionless), private, consortium, and hybrid types. Public blockchains like Bitcoin and Ethereum allow anyone to join, validate transactions, and participate in consensus without central authority.

Mining and Its Role

Mining is the process by which new blocks are created and added to the blockchain. In proof-of-work (PoW) systems like Bitcoin, miners compete to solve a computationally intensive cryptographic puzzle based on the SHA-256 algorithm. The first miner to find a valid solution—producing a hash below a target value—broadcasts the block to the network for validation.

Mining serves two critical purposes:

Securing the network against double-spending and fraud.
Introducing new cryptocurrency into circulation through block rewards.
The difficulty of the puzzle adjusts every 2,016 blocks (approximately every two weeks in Bitcoin) to maintain an average block time of 10 minutes.

Multi-Node Blockchain Network

A blockchain network consists of nodes, each maintaining a copy of the entire ledger. Nodes can be classified as:

Full nodes: Store the complete blockchain and validate transactions independently.
Light nodes: Store only block headers, relying on full nodes for data.
Mining nodes (miners): Participate in block creation and validation.
Super nodes: Relay information between nodes to ensure network synchronization.

All nodes enforce consensus rules, ensuring agreement on the state of the blockchain. When a new block is mined, it is propagated across the network, and each node verifies its validity before appending it to their local copy.

Consensus Mechanisms

Consensus mechanisms ensure all nodes agree on the validity of transactions and the state of the blockchain. The most common types include:

Proof of Work (PoW)

Used by Bitcoin, PoW requires miners to expend computational effort solving puzzles. The longest chain (representing the most work) is accepted as canonical. Security relies on the assumption that honest nodes control more than 50% of the network's computing power.

Proof of Stake (PoS)

Used by Ethereum post-"Merge", PoS selects validators based on the amount of cryptocurrency they "stake" as collateral. Validators propose and attest to blocks, earning rewards in return. This method is significantly more energy-efficient than PoW.

Other variants include Delegated Proof of Stake (DPoS), Practical Byzantine Fault Tolerance (PBFT), and hybrid models.

Transactions and Mining Reward

When a user initiates a transaction (e.g., sending bitcoin), it is broadcast to the network and held in a mempool (memory pool) until picked up by a miner. Miners bundle multiple transactions into a block and attempt to mine it.

Miners are incentivized through two types of rewards:

Block reward (coinbase reward): Newly minted cryptocurrency (e.g., currently 6.25 BTC per block in Bitcoin, halving approximately every four years).

Transaction fees: Paid by users to prioritize their transactions.

As block rewards decrease over time, transaction fees are expected to become the primary income source for miners. After 2140, when no new bitcoins will be created, mining will be sustained solely by transaction fees.

Chain Replacement and Fork Resolution

When two miners solve a block at nearly the same time, a temporary fork occurs, resulting in two competing chains. Nodes continue building on whichever chain they receive first. The conflict is resolved through the longest chain rule: the chain with the most accumulated proof-of-work becomes the accepted version.

Code :-

```
class Blockchain:
    def __init__(self):
        self.chain = []
        self.transactions = []
        self.create_block(proof=1, previous_hash='0')
        self.nodes = set()
    def create_block(self, proof, previous_hash):
        block = {
            'index': len(self.chain) + 1,




        'timestamp': str(datetime.datetime.now()),
        'proof': proof,
        'previous_hash': previous_hash,
        'transactions': self.transactions
    }
    self.transactions = []
    self.chain.append(block)
    return block

def get_previous_block(self):
    return self.chain[-1]

def proof_of_work(self, previous_proof):
    new_proof = 1
    check_proof = False
    while not check_proof:
        hash_operation = hashlib.sha256(
            str(new_proof**2 - previous_proof**2).encode()
        ).hexdigest()
        if hash_operation[:4] == '0000':
            check_proof = True
        else:
            new_proof += 1
    return new_proof
```

```
def hash(self, block):
    encoded_block = json.dumps(block, sort_keys=True).encode()
    return hashlib.sha256(encoded_block).hexdigest()

def is_chain_valid(self, chain):
    previous_block = chain[0]
    block_index = 1
    while block_index < len(chain):
        block = chain[block_index]
        if block['previous_hash'] != self.hash(previous_block):
            return False
        previous_proof = previous_block['proof']
        proof = block['proof']
        hash_operation = hashlib.sha256(
            str(proof**2 - previous_proof**2).encode()



          ).hexdigest()
          if hash_operation[:4] != '0000':
              return False
          previous_block = block
          block_index += 1
      return True

  def add_transaction(self, sender, receiver, amount):
      self.transactions.append({
          'sender': sender,
          'receiver': receiver,
          'amount': amount
      })
      previous_block = self.get_previous_block()
      return previous_block['index'] + 1

  def add_node(self, address):
      parsed_url = urlparse(address)
      self.nodes.add(parsed_url.netloc)



   def replace_chain(self):
       network = self.nodes
       longest_chain = None
       max_length = len(self.chain)
       for node in network:
           response = requests.get(f'http://{node}/get_chain')
           if response.status_code == 200:
               length = response.json()['length']
               chain = response.json()['chain']
               if length > max_length and self.is_chain_valid(chain):
                   max_length = length
                   longest_chain = chain
       if longest_chain:
           self.chain = longest_chain
           return True
       return False

app = Flask(__name__)

node_address = str(uuid4()).replace('-', '')
```

```
blockchain = Blockchain()

@app.route('/mine_block', methods=['GET'])
def mine_block():
    previous_block = blockchain.get_previous_block()
    previous_proof = previous_block['proof']
    proof = blockchain.proof_of_work(previous_proof)
    previous_hash = blockchain.hash(previous_block)
    blockchain.add_transaction(
        sender=node_address,
        receiver='Richard',
        amount=1
    )
    block = blockchain.create_block(proof, previous_hash)
    response = {
        'message': 'Congratulations, you just mined a block!',
        'index': block['index'],
        'timestamp': block['timestamp'],
        'proof': block['proof'],
        'previous_hash': block['previous_hash'],
        'transactions': block['transactions']
    }
    return jsonify(response), 200
```

```
@app.route('/get_chain', methods=['GET'])
def get_chain():
    response = {
        'chain': blockchain.chain,
        'length': len(blockchain.chain)
    }
    return jsonify(response), 200

@app.route('/is_valid', methods=['GET'])
def is_valid():
    is_valid = blockchain.is_chain_valid(blockchain.chain)
    if is_valid:
        response = {'message': 'All good. The Blockchain is valid.'}
    else:
        response = {'message': 'The Blockchain is not valid.'}
```

```
        return jsonify(response), 200

    @app.route('/add_transaction', methods=['POST'])
    def add_transaction():
        json_data = request.get_json()
        transaction_keys = ['sender', 'receiver', 'amount']
        if not all(key in json_data for key in transaction_keys):
            return 'Some elements of the transaction are missing', 400
        index = blockchain.add_transaction(
            json_data['sender'],
            json_data['receiver'],
            json_data['amount']
        )
        response = {
            'message': f'This transaction will be added to Block {index}'
        }
        return jsonify(response), 201

    @app.route('/connect_node', methods=['POST'])
    def connect_node():
        json_data = request.get_json()
        nodes = json_data.get('nodes')
        if nodes is None:
            return "No node", 400
        for node in nodes:
            blockchain.add_node(node)
        response = {
            'message': 'All the nodes are now connected.',
            'total_nodes': list(blockchain.nodes)
        }
        return jsonify(response), 201

    @app.route('/replace_chain', methods=['GET'])
    def replace_chain():
        is_chain_replaced = blockchain.replace_chain()
        if is_chain_replaced:
            response = {
                'message': 'The chain was replaced by the longest one.',
                'new_chain': blockchain.chain
            }


     else:
         response = {
             'message': 'The chain is already the largest one.',
             'actual_chain': blockchain.chain
         }
     return jsonify(response), 200

app.run(host='0.0.0.0', port=5000)
```

D20A 59 Jai Talreja Blockchain lab

Output :-

GET echo GET

Anchor tags in Postman documentation / GET Requests / echo GET

GET http://127.0.0.1:5000/mine_block

Overview Params Authorization Headers (7) Body Scripts Settings

Headers 7 hidden

Key	Value

Body Cookies Headers (5) Test Results

{} JSON Preview Visualize


```
{
    "block": {
        "index": 3,
        "previous_hash": "4d56ca2f3e090aa136cbc8e9c40c5b4154438c7ed33a85fd9461e8c53d2afe04",
        "proof": 45293,
        "timestamp": "2026-02-03 09:29:02.261406",
        "transactions": [
            {
                "amount": 1,
                "receiver": "Miner",
                "sender": "f50c937a472445029581f4c4dd5626fd"
            }
        ]
    },
    "message": "Block mined successfully"
}
```


GET echo GET

Anchor tags in Postman documentation / GET Requests / echo GET

GET http://127.0.0.1:5000/get_chain

Overview Params Authorization Headers (7) Body Scripts Settings

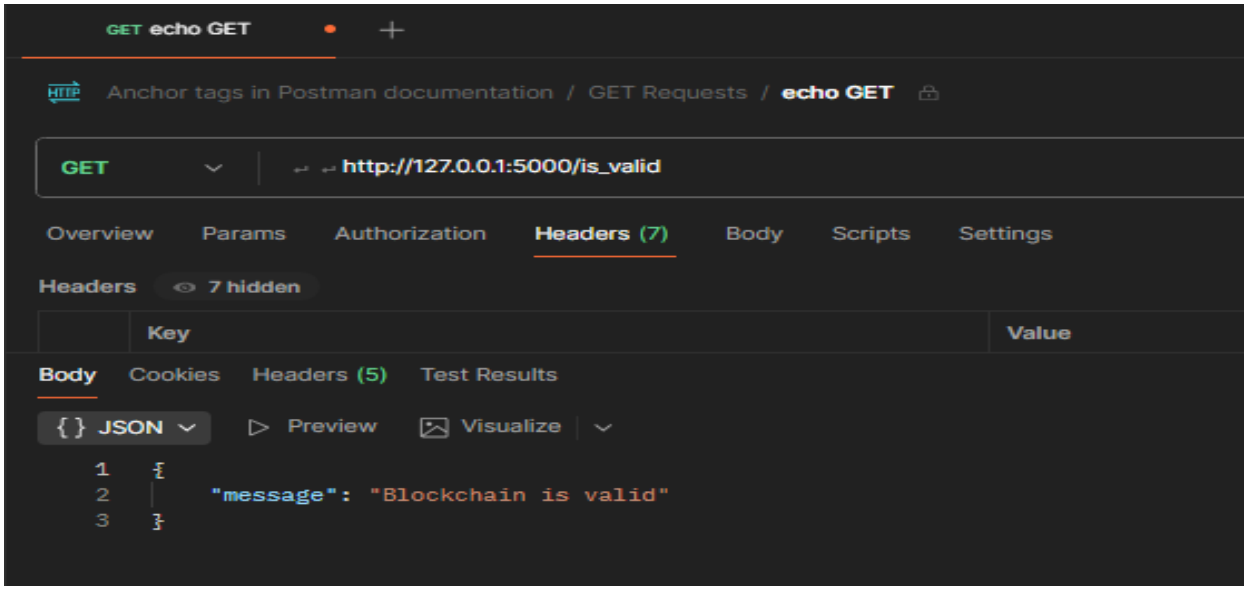
Headers 7 hidden

Key	Value
Key	Value

Body Cookies Headers (5) Test Results

{} JSON Preview Visualize


```
{
    "chain": [
        {
            "index": 1,
            "previous_hash": "0",
            "proof": 1,
            "timestamp": "2026-02-03 09:19:02.441105",
            "transactions": []
        },
        {
            "index": 2,
            "previous_hash": "badca02819f0ebae886535c28b2aef8affbd61def0f9537257cb84f22913acd7",
            "proof": 533,
            "timestamp": "2026-02-03 09:28:30.853839",
            "transactions": [
                {
                    "amount": 1,
                    "receiver": "Miner",
                    "sender": "f50c937a472445029581f4c4dd5626fd"
                }
            ]
        },
        {
```



Conclusion :-

This implementation demonstrates how blockchain technology works by creating a decentralized ledger using Python. The code shows how blocks are securely linked using cryptographic hashes, how transactions are recorded, and how Proof-of-Work is used to mine new blocks. The use of consensus through chain validation and replacement ensures data integrity and synchronization across nodes. Overall, the system highlights the core principles of blockchain such as decentralization, transparency, security, and immutability.